AURA

Claude Code Engineering Playbook

& Scalable Project Blueprint

Document title	AURA — Claude Code Engineering Playbook
Companion to	AURA SDLC Documentation v1.0
Version	1.0
Date	22 June 2026
Author / Owner	Anurag Kumar Singh
Basis	Claude Code mastery notes (CampusX) + AURA architecture
Classification	Confidential

This playbook documents how AURA is engineered end to end using Claude Code: project memory, spec-driven development, skills, subagents, hooks, MCP, CI/CD automation and security — concluding with the complete scalable project structure.

Table of Contents

CHAPTER 1

How AURA Is Built With Claude Code

AURA is developed entirely inside Claude Code, the command-line agentic coding environment. Rather than writing every line by hand, the developer works as a director: stating intent, reviewing plans, and approving changes, while Claude Code reads the project, edits files, runs tests and commits — always under human control. This playbook turns that loose idea into a disciplined, repeatable engineering process suited to a security-sensitive, scalable product.

1.1 The agentic loop, applied to AURA

Claude Code operates a read-plan-act-verify loop: it gathers context from the codebase, proposes a plan, uses tools to make changes, then checks its own work against tests and lint. AURA's own runtime mirrors this same discipline, so the way the product is built and the way it behaves share one philosophy: plan, act on the smallest safe step, observe the result, and keep a human in the loop for anything risky.

1.2 Control surfaces used throughout

Capability	Role in building AURA
CLAUDE.md	Project memory: architecture, rules, commands Claude always respects
Spec-driven dev	Every feature starts as a written, reviewed spec before code
Plan / permission modes	Claude proposes before touching code; you approve the gear
Skills	One-word commands for AURA's repeated tasks (add a tool, review security)
Subagents	Specialised, parallel workers (test author, security reviewer)
Hooks	Deterministic quality and safety gates on every edit and commit
MCP	Connects Claude to GitHub, Gmail and other systems during development
CI/CD headless	claude -p runs reviews and tests automatically in the pipeline

CHAPTER 2

Project Memory — The AURA CLAUDE.md

CLAUDE.md is the file Claude Code reads automatically at the start of every session. It prevents the agent from guessing conventions or running the wrong commands. For AURA it encodes the architecture, the non-negotiable safety rules, and the exact build commands. The example below follows the recommended eight-section structure.

```
# AURA — Project Overview
Local-first, voice + face desktop agent (a practical Jarvis). Python 3.10+.
Built on SOLID: every external dependency sits behind an interface; the
orchestrator depends only on interfaces, concretes injected by factories.

# Key Directories
src/core/      - config, logging, shared types
src/io/        - asr (Listener), tts (Speaker), wake (WakeDetector)
src/auth/      - Authenticator (face verification gate)
src/llm/       - LLMProvider (litellm: Gemini Flash/Pro, local Ollama)
src/agent/     - planner: plan-act-observe-replan loop
src/tools/     - Tool + ToolRegistry (open_app, send_email, run_command, read_screen)
src/security/  - ConfirmationPolicy (risk + identity + confirm)
src/factories/ - build concrete objects from Settings
tests/         - pytest; the LLM is faked, no network

# Commands
python main.py                # run the agent (console mode)
pytest -q                     # run all tests
ruff check .                  # lint

# Architecture Rules (NON-NEGOTIABLE)
- Never import a concrete library into core/ or agent/. Depend on interfaces.
- Add a tool or model provider by REGISTERING it; never edit the planner.
- Every Tool returns a ToolResult. Failures are data, not exceptions.

# Safety Rules (NON-NEGOTIABLE)
- All non-SAFE actions pass through ConfirmationPolicy: identity + confirm.
- Never weaken, bypass or auto-approve the confirmation gate.
- Secrets come from env only. Never log or commit a key.

# Testing
- New logic needs a test. The LLM must be faked (see tests/conftest.py).
- pytest and ruff must pass before any commit.

# Style
- PEP 8, type hints on public functions, docstrings explain WHY.

# What NOT to do
- Do not add cloud calls to local mode. Do not screen-scrape where an API exists.
```

2.1 Path-scoped rules

Security-critical folders get extra, automatically-loaded rules via .claude/rules/. A rule scoped to src/security/ reminds Claude on every edit there that the confirmation gate must never be relaxed — the instruction loads only when those files are touched, keeping context lean.

```
# .claude/rules/security.md  (path: "src/security/**")
This folder is the safety core of AURA.
- Do NOT change the meaning of risk tiers without an approved spec.
- Sensitive and destructive actions ALWAYS require identity + explicit confirm.
- Add a regression test for every change here.
```

CHAPTER 3

Spec-Driven Development for AURA Features

“Vibe coding” — describing a feature loosely and hoping — fails on a system with real safety stakes. AURA uses Spec-Driven Development: every feature begins as a short written specification that is reviewed and approved before a line of code is generated. The spec is the contract; Claude Code implements against it and the reviewer checks the result against it.

3.1 The workflow

- Research: Claude reads the relevant code and summarises how it works today.
- Spec: write specs/NN-feature.md — goal, scope, interfaces, risks, test plan.
- Plan: Claude proposes a step-by-step plan in Plan Mode; you approve.
- Implement: Claude edits files, runs tests and lint, iterating until green.
- Review: a security/quality subagent checks the diff against the spec.

3.2 Example spec — Gmail send_email tool

```
# specs/04-gmail-send-email.md

## Goal
Make the send_email tool actually send via the Gmail API (currently a stub).

## Scope
- IN:  OAuth (one-time), token cached under ~/.aura, send a plain-text email.
- OUT: attachments, HTML email, multiple accounts (later specs).

## Interfaces / constraints
- Keep Tool.run signature and ToolResult return unchanged.
- risk = SENSITIVE -> stays behind ConfirmationPolicy. Do NOT change this.
- Secrets/token from local store only; never logged (logging redaction applies).

## Risks
- Token leakage -> store with restricted file permissions; never print.
- Wrong recipient -> confirmation message must show the resolved 'to' address.

## Test plan
- Unit: mock the Gmail client; assert payload built correctly.
- Security: assert send is blocked without confirmation and without identity.
- TC mapping: TC-2 (sensitive requires confirm), TC-5 (identity gate).

## Definition of Done
pytest + ruff green; spec linked in PR; reviewer subagent approves.
```

CHAPTER 4

Skills — One-Word Commands for AURA

Skills are reusable instructions invoked like /command. They capture AURA's repeated workflows so the developer never re-types a long prompt. Skills live in .claude/skills/ and each is a SKILL.md with frontmatter plus instructions.

4.1 /add-tool — scaffold a new tool correctly

```
# .claude/skills/add-tool/SKILL.md
---
name: add-tool
description: Scaffold a new AURA Tool with correct risk tier, registration and test.
---
When invoked with a tool name and purpose:
1. Create src/tools/NAME.py implementing Tool: name, description, risk, parameters(), run().
2. run() must return ToolResult (ok, summary, data, error) — never raise for expected failures.
3. Choose the risk tier deliberately: SAFE (read-only), SENSITIVE (sends/changes),
   DESTRUCTIVE (irreversible/shell). When unsure, choose the higher tier.
4. Register it in src/factories/component_factory.py build_tools().
5. Add tests/test_NAME.py covering success, failure, and (if non-SAFE) the confirm gate.
6. Run pytest and ruff; fix until green. Reference the spec id in the summary.
```

4.2 /security-review — invariant check

```
# .claude/skills/security-review/SKILL.md
---
name: security-review
description: Audit a change for AURA's safety and secret-handling invariants.
---
Review the current diff and report violations of:
1. Confirmation gate: any non-SAFE action that can run without identity + confirm.
2. Risk tiers: a sensitive/destructive action mislabelled as SAFE.
3. Secrets: any key read from outside env, or any path that could log a secret.
4. Shell: any command execution outside the allow-list.
5. Tests: missing regression test for security-relevant logic.
Output a short PASS/FAIL with file:line references. Do not edit code.
```

CHAPTER 5

Subagents — Specialised & Parallel Workers

Subagents are separate Claude instances with their own context, used to keep the main session lean and to run specialised or parallel work. AURA defines a small team: a security reviewer, a test author, and a read-only quality reviewer. Each is a markdown file in .claude/agents/ with configuration frontmatter.

5.1 Security reviewer (read-only)

```
# .claude/agents/aura-security-reviewer.md
---
name: aura-security-reviewer
description: Reviews diffs for AURA safety invariants. Use after any change to
  src/security, src/tools, or src/auth.
tools: Read, Glob, Grep        # read-only: cannot edit
model: inherit
permissionMode: plan
maxTurns: 20
---
You are AURA's security reviewer. Verify the confirmation gate is intact, risk
tiers are correct, secrets stay in env, and shell stays allow-listed. Report
PASS/FAIL with file:line evidence. Never modify files.
```

5.2 Test author

```
# .claude/agents/aura-test-author.md
---
name: aura-test-author
description: Writes pytest tests for new AURA logic with the LLM faked.
tools: Read, Glob, Grep, Edit, Bash
model: inherit
permissionMode: acceptEdits
maxTurns: 30
skills: [add-tool]
---
Write focused tests that fake the LLM (see tests/conftest.py), cover success,
failure and security-gate paths, and leave pytest green. No network in tests.
```

5.3 Parallel and isolated work

For broad refactors, several subagents run in parallel, each in its own git worktree (isolation: worktree) so they never collide. The main session stays focused while specialists work independently and report back.

CHAPTER 6

Hooks — Deterministic Quality & Safety Gates

Hooks are scripts Claude Code runs automatically at lifecycle events. Unlike instructions, they are deterministic — they always fire. AURA uses hooks to lint after every edit, to run tests before any commit, and to protect the security core. The PreToolUse event can block an action entirely (exit code 2).

6.1 hooks.json

```
// .claude/hooks.json
{
  "hooks": {
    "PostToolUse": [
      { "matcher": "Edit|Write",
        "command": "ruff check --fix $CLAUDE_FILE &amp;&amp; ruff format $CLAUDE_FILE" }
    ],
    "PreToolUse": [
      { "matcher": "Bash(git commit*)",
        "command": "pytest -q || exit 2" }
    ],
    "PostCompact": [
      { "command": "cat .claude/rules/security.md" }
    ]
  }
}
```

How the gates behave

- PostToolUse on Edit/Write: every file Claude saves is auto-linted and formatted.
- PreToolUse on git commit: tests run first; a non-zero exit (code 2) blocks the commit.
- PostCompact: the safety rules are re-injected after the context is compressed, so Claude never ‘forgets’ them mid-session.

6.2 Choosing the right tool

Need	Use
Always-on, deterministic enforcement	Hook
Reusable instruction invoked on demand	Skill
Persistent project knowledge and rules	CLAUDE.md
Specialised or parallel reasoning	Subagent

CHAPTER 7

MCP, CI/CD, Security & Cost

7.1 MCP — connecting external systems

The Model Context Protocol lets Claude Code talk to external systems through a standard interface. During development, AURA connects to GitHub (issues, PRs) and, when building the email feature, to Gmail. Servers are declared in .mcp.json and shared with the team.

```
// .mcp.json
{
  "mcpServers": {
    "github": { "command": "npx", "args": ["-y", "@modelcontextprotocol/server-github"] },
    "gmail":  { "command": "npx", "args": ["-y", "server-gmail"] }
  }
}
```

7.2 CI/CD — headless Claude in the pipeline

In automation, Claude runs headlessly with claude -p (print mode): it executes one instruction and exits, perfect for a pipeline step. AURA uses it to review every pull request for safety-invariant violations before a human looks.

```
# .github/workflows/review.yml  (excerpt)
- name: AURA security review
  run: |
    claude -p "Run the security-review skill on this PR diff. \
    Fail if the confirmation gate, risk tiers, or secret handling are violated." \
    --output-format stream-json
```

7.3 Security when building with Claude Code

- Prompt injection: treat issue text, web content and screen captures as untrusted; they must never auto-trigger actions. This mirrors AURA's own runtime rule.
- Permissions: allow-list safe commands; require approval for the rest; deny destructive patterns.
- Sandboxing: restrict filesystem and network scope so the agent cannot reach beyond the project.
- Zero Data Retention is available for sensitive work where prompts must not be retained.

7.4 Cost and context discipline

- Use a smaller, faster model for routine edits; reserve the strongest model for hard design work.
- Keep sessions lean: /clear between unrelated tasks, /compact when long, subagents for big reads.
- A lean context is also the biggest cost saver — most spend is tokens re-read every turn.

CHAPTER 8

Expected Scalable Project Structure

The structure below is what AURA looks like when built professionally with Claude Code. It has two halves: the application itself (src/, tests/, packaging) and the Claude Code engineering layer (.claude/, CLAUDE.md, .mcp.json, CI). Both grow without disturbing the core, because new tools, providers, skills and agents are added by placing a file, not by editing central logic.

```
aura/
|
|-- CLAUDE.md                      # project memory (auto-loaded every session)
|-- CLAUDE.local.md                # private, gitignored personal notes
|-- README.md
|-- LICENSE
|-- requirements.txt
|-- pyproject.toml                 # packaging + tool config (ruff, pytest)
|-- .env.example                  # config template (real .env is gitignored)
|-- .gitignore
|-- main.py                        # thin entry point
|
|-- .claude/                       # === Claude Code engineering layer ===
|   |-- settings.json             # shared team settings
|   |-- hooks.json                # quality + safety gates (lint, test-before-commit)
|   |-- commands/                 # legacy custom slash commands (optional)
|   |-- skills/                   # one-word workflows
|   |   |-- add-tool/SKILL.md
|   |   |-- security-review/SKILL.md
|   |   \-- release/SKILL.md
|   |-- agents/                   # subagents (specialised workers)
|   |   |-- aura-security-reviewer.md
|   |   |-- aura-test-author.md
|   |   \-- aura-quality-reviewer.md
|   |-- rules/                    # path-scoped rules (auto-load by folder)
|   |   |-- security.md           # scoped to src/security/**
|   |   \-- tools.md              # scoped to src/tools/**
|   \-- specs/                    # spec-driven development docs
|       |-- 01-core-loop.md
|       |-- 02-voice-io.md
|       |-- 03-face-auth.md
|       \-- 04-gmail-send-email.md
|
|-- .mcp.json                      # MCP servers (github, gmail)
|
|-- src/                           # === application (SOLID layers) ===
|   |-- __init__.py
|   |-- orchestrator.py           # wires layers, runs top-level loop
|   |-- core/                     # config, logging, shared types
|   |   |-- config.py
|   |   |-- logging.py
|   |   \-- types.py
|   |-- io/                       # human I/O boundary
|   |   |-- interfaces.py
|   |   |-- asr/                  # Listener: console, whisper, deepgram
|   |   |-- tts/                  # Speaker: console, piper, elevenlabs
|   |   \-- wake/                 # WakeDetector
|   |-- auth/                     # Authenticator (face gate)
|   |-- llm/                      # LLMProvider (litellm: gemini flash/pro/local)
|   |-- agent/                    # planner: plan-act-observe-replan
|   |-- tools/                    # Tool + ToolRegistry + concrete tools
|   |   |-- base.py
|   |   |-- registry.py
|   |   |-- open_app.py
|   |   |-- send_email.py
|   |   |-- run_command.py
|   |   \-- read_screen.py
```

```
|   |-- security/                # ConfirmationPolicy (the safety core)
|   |-- memory/                  # ConversationMemory (in-memory, sqlite)
|   \-- factories/               # build concretes from Settings (composition root)
|
|-- tests/                        # pytest; LLM faked, no network
|   |-- conftest.py
|   |-- test_policy.py
|   |-- test_planner_loop.py
|   \-- test_tools/
|
|-- evals/                        # agent reliability harness + task suite
|   |-- tasks.jsonl
|   \-- run_eval.py
|
|-- scripts/                      # verify_setup, oauth, packaging helpers
|-- assets/                       # icons, enrolled-face store path, models
|-- docs/                         # SDLC docs, this playbook, ADRs
|   \-- adr/
|       |-- ADR-001-interfaces-and-di.md
|       \-- ADR-002-single-confirmation-policy.md
|
\-- .github/
    \-- workflows/
        |-- ci.yml               # ruff + pytest on every push
        \-- review.yml           # claude -p security review on PRs
```

8.1 Why this scales

- New capability = new file: a tool, provider, skill or agent is added by dropping a file in its folder; central logic is never touched (Open/Closed).
- Engineering rules travel with the code: CLAUDE.md and .claude/rules keep every contributor (human or Claude) aligned automatically.
- Safety is structural: the confirmation core is one small module, guarded by a path-scoped rule, a review subagent, a hook, and a CI check.
- Reliability is measured: the evals/ harness turns the audit log into a regression dataset.
- Packaging is first-class: pyproject.toml plus scripts/ produce signed desktop installers.

CHAPTER 9

Build Roadmap Mapped to Claude Code

Each milestone from the SDLC plan is delivered using a specific set of Claude Code capabilities, in a consistent rhythm: write the spec, plan, implement with hooks enforcing quality, review with a subagent, and let CI guard the merge.

Milestone	Primary Claude Code capabilities
M1 Core loop	CLAUDE.md set up; spec 01; Plan Mode; test-before-commit hook
M2 Voice I/O	spec 02; add-tool skill pattern for providers; ruff hook
M3 Face auth	spec 03; security.md rule; security-reviewer subagent
M4 Reach (email/web/screen)	MCP (gmail, github); spec 04; parallel subagents in worktrees
M5 Hardening	evals harness; security-review skill; CI review.yml
M6 Release	release skill; pyproject packaging; signed installers

9.1 The per-feature rhythm

- Research the existing code (Claude summarises; keeps context lean with subagents).
- Write specs/NN-feature.md and have it reviewed.
- Plan Mode: approve the step-by-step plan before any edit.
- Implement; hooks lint each save and block commits if tests fail.
- Security-reviewer subagent checks the diff; CI repeats the check on the PR.
- Merge, tag a semantic version, update docs and the traceability matrix.

APPENDIX

Claude Code Capability Cheat Sheet

Capability	What it is	AURA usage
CLAUDE.md	Auto-loaded project memory	Architecture + safety rules + commands
Rules (.claude/rules)	Path-scoped instructions	Extra guard on src/security and src/tools
Spec-driven dev	Write the spec before code	specs/NN-*.md per feature
Plan Mode	Propose before acting	Approve plans for risky changes
Skills	/command workflows	add-tool, security-review, release
Subagents	Separate context workers	security reviewer, test author, quality
Hooks	Deterministic event scripts	lint on edit, tests before commit
MCP	External system connectors	github, gmail during development
Headless (claude -p)	Scriptable one-shot runs	CI security review on PRs
Worktrees	Isolated git sandboxes	Parallel refactors without conflict
Permission modes	Control autonomy level	Plan/acceptEdits per task and per agent

This playbook is a companion to the AURA SDLC Documentation. Together they cover what is being built and exactly how it is engineered with Claude Code, from first spec to signed release.